

Enhancing Microservices through Cross-Service Logic Duplication Resolution and Intelligent Function Relocation Detection

T.B.A. Jayadewa
Index No: 21000859

P.N. Jayasuriya
Index No: 21000875

K.J.A.L. Randika
Index No: 21001596

Supervisor: Dr. Chaminda Ranasinghe

Co-Supervisor: Dr. Chamath Keppetiyagama



University of Colombo School of Computing

Colombo, Sri Lanka

Submitted in partial fulfillment of the requirements of the
B.Sc(Hons) in Software Engineering 4th Year Project (SCS4223)

April 2026

Contents

1	Introduction	2
1.1	Background	2
1.2	Research Problem	3
1.3	Research Aim	5
1.4	Research Objectives	5
1.5	Research Questions	6
1.6	Project Scope	6
1.6.1	In-Scope Functionalities	6
1.6.2	Out-of-Scope Functionalities	6
1.7	Significance of the Project	7
2	Literature Review	9
2.1	Review of Existing Solutions for Microservice Optimization	9
2.2	Limitations, inconsistencies, or opportunities identified by prior work	11
3	Methodology and System Design	14
3.1	Semantic Logic Duplication Detection	14
3.1.1	Service & Code Collection via Backstage	15
3.1.2	Function-Level Extraction	16
3.1.3	Data Flow Graph Extraction and Slicing	17
3.1.3.1	Data Flow Graph Extraction	17
3.1.3.2	Slicing	18
3.1.4	Fine-Tuning GraphCodeBERT for Semantic Clone Detection	18
3.1.4.1	Overview and Motivation	18
3.1.4.2	Dataset	20
3.1.4.3	Model Architecture and Task Formulation	20
3.1.4.4	Hyperparameter Configuration	21

3.1.4.5	Training Procedure and Input Representation . . .	21
3.1.5	Cross-Service Similarity Computation	22
3.2	Function Relocation Analyzer	22
3.2.1	Overview	22
3.2.2	Phase 1 – Data Collection	23
3.2.3	Phase 2 – Trace Preprocessing	24
3.2.4	Phase 3 – Function Call Analysis	25
3.2.5	Phase 4 – Function Relocation Decision Logic	26
3.2.6	Phase 5 – Validation and Reporting	27
3.3	Refactoring Support Plugin	28
3.3.1	Logic Duplication Detection Plugin	28
3.3.2	Function Relocation Analyzer Plugin	29
4	Experiments	30
4.1	Logic Duplication Detector	30
4.1.1	Experiment 1 - CodeBERT Baseline: File-Level Text Simi- larity	30
4.1.1.1	Experimental Objective	30
4.1.1.2	Experimental Setup	30
4.1.1.3	Results	31
4.1.1.4	Analysis and Conclusion	31
4.1.2	Experiment 2 - Joern CPG Pipeline: Feasibility Assessment	32
4.1.2.1	Experimental Objective	32
4.1.2.2	Experimental Setup	32
4.1.2.3	Results and Finding	32
4.1.2.4	Conclusion and Decision	33
4.1.3	Experiment 3 - Zero-Shot GraphCodeBERT: BigCloneBench Evaluation	33
4.1.3.1	Experimental Objective	33
4.1.3.2	Experimental Setup	33
4.1.3.3	Results	34

4.1.3.4	Analysis and Conclusion	34
4.1.4	Experiment 4 - Zero-Shot GraphCodeBERT with DFG: Thresh- old Calibration on BigCloneBench	35
4.1.4.1	Experimental Objective	35
4.1.4.2	Experimental Setup	35
4.1.4.3	Results and Conclusion	36
4.1.4.4	Analysis	37
4.1.4.5	Conclusion	37
4.1.5	Experiment 5 - Zero-Shot Cross Language Detection on the Google Online Boutique	38
4.1.5.1	Experimental Objective	38
4.1.5.2	Experimental Setup	38
4.1.5.3	Results	39
4.1.5.4	Analysis	39
4.1.5.5	Conclusion	41
4.1.6	Experiment 6 - Controlled Evaluation on the Synthetic Poly- glot Ecommerce Repository	43
4.1.6.1	Experimental Objective	43
4.1.6.2	Experimental Setup	43
4.1.6.3	Results	43
4.1.6.4	Analysis	45
4.1.6.5	Conclusion	45
4.1.7	Experiment 7 - Fine-Tuned GraphCodeBERT on Unsliced BigCloneBench Subset	46
4.1.7.1	Experimental Objective	46
4.1.7.2	Experimental Setup	46
4.1.7.3	Results	47
4.1.7.4	Analysis and Conclusion	47
4.1.8	Experiment 8 - Fine-Tuned GraphCodeBERT on DFG-Sliced BigCloneBench Subset	48

4.1.8.1	Experimental Objective	48
4.1.8.2	Experimental Setup	48
4.1.8.3	Results	49
4.1.8.4	Analysis and Conclusion	49
4.2	Function Relocation Analyzer	50
4.2.1	Experiment 1 - Evaluation of Distributed Tracing Integra- tion Strategies	50
4.2.1.1	Experimental Objective	50
4.2.1.2	Experimental Setup	50
4.2.1.3	Experimental Procedure	51
4.2.1.4	Results	54
4.2.1.5	Adopted Configuration	54
4.2.1.6	Discussion and Conclusion	55
5	Implementation	57
6	Evaluation	58

Chapter 1

Introduction

1.1. Background

Monolithic architecture has traditionally been the standard approach to software development, where the entire application is built as a single, unified unit. While it offers simplicity in the early stages of development, it presents several challenges as the system grows, including limited scalability, difficulty in maintenance, tight coupling, and obstacles to continuous deployment (Noor et al. 2024).

To address these limitations, the Microservices Architecture (MSA) emerged as a modern alternative. MSA structures software applications as a collection of small, independently deployable services, each running its own process and communicating through lightweight mechanisms such as HTTP (Lewis & Fowler 2014). These services can be developed using various technologies that are suited to particular needs, and they are structured around distinct business capabilities. Because of this flexibility, each service can use the best technology for its functional needs. Changes to one service can be made without impacting others thanks to this independence, which promotes high cohesiveness and loose coupling, both of which increase system flexibility and agility (Thönes 2015). Because failures in one service do not always impact others, microservices also improve system resilience. Isolating failures improves overall system reliability, as Sam Newman points out in *Building Microservices*. Microservices also make it possible to scale individual services separately rather than the entire application, which enhances system performance and resource utilization (Newman 2019).

Due to these benefits, MSA has become widely adopted across the software industry. Many organizations are actively migrating from monolithic legacy sys-

tems to MSA in pursuit of improved performance, maintainability, and responsiveness to change. However, this shift also introduces new challenges, including increased complexity in system design and the need for better optimization strategies (Nogueira et al. 2024).

This research is motivated by the growing need to optimize microservices-based systems, particularly in the areas of logic reuse, relocation of misaligned functionalities, and automated refactoring. By addressing these concerns, the project aims to support more efficient development and evolution of microservice architectures in real-world software systems.

1.2. Research Problem

In microservice-based systems, an ongoing and often overlooked issue is the accidental duplication of code across different services (Mo et al. 2021). This problem arises due to the decentralized nature of development—independent teams which often build and maintain services in isolation. While this promotes autonomy and rapid delivery, it also increases the risk of duplicating functionalities when similar business logic is unknowingly implemented by different teams. Over time, this leads to inconsistencies, elevated maintenance costs, and reduced overall code quality.

Such duplication weakens the modularity and maintainability that microservices are meant to provide. It complicates cross-service integration, causes redundant development efforts, and contributes to technical debt. Although organizations attempt to mitigate this issue using shared libraries or service reuse, these solutions are typically applied reactively and lack systematic support (Nogueira et al. 2024).

As microservices evolve, there might be a gradual drift in service responsibilities, leading to higher affinity among functions in different services. This often results in functions residing in services where they no longer belong, either because their usage context has changed or because they are now more relevant to another domain or service. In such cases, it might be more efficient to colocate these

tightly coupled functions within a single service (Panichella et al. 2021). While developers may recognize these issues, manually identifying and relocating such functions is error-prone and time-consuming, especially in large codebases. Consequently, there is a clear need for automated or semi-automated solutions that can analyze function usage patterns, assess function dependencies, and suggest or support appropriate function relocation across services.

Existing static analysis tools are typically designed for monolithic systems or limited to analyzing individual repositories, making them ineffective at detecting logic duplication across multiple microservices. Additionally, there is a lack of support for recommending function relocation based on usage patterns or service coupling. Moreover, there is a shortage of refactoring tools that can assist developers in managing semantic code duplication and function relocation across services (Akram et al. 2018). Furthermore, empirical studies and practical tools that offer architectural-level refactoring guidance are limited, leaving developers without effective means to improve and maintain the overall quality of microservice architectures.

This qualifies as a software engineering research problem because it:

- Directly pertains to software architecture optimization, a core area of software engineering.
- Involves improving code quality, maintainability, and modular design—fundamental concerns in the field.
- Requires novel or enhanced techniques in logic duplication detection, function relocation, and refactoring support.

Known Gaps Addressed:

- Absence of cross-service semantic logic detection in existing static analysis tools, which are mostly monolithic or repo-bound.
- Lack of function relocation recommendations based on service usage patterns or dependency analysis.

- Limited research and tool support for cross-service refactoring guidance in microservice systems.

1.3. Research Aim

This research aims to improve the maintainability, scalability, and modularity of microservices architecture by automatically identifying logic duplication across services and mispositioned functions in distributed systems. Moreover, this research is focused on offering developers clear and precise refactoring suggestions to help them make appropriate decisions while refactoring the boundaries of their services. By addressing problems related to unwanted coupling, logic duplication, and function misposition, this research will be beneficial in reducing the overall complexity and improving the sustainability of microservices-based applications.

1.4. Research Objectives

The primary objective of this research is to enhance the maintainability and architectural quality of microservice-based systems by addressing cross-service logic duplication and function misplacement. To achieve this, the research will pursue the following specific objectives:

1. **RO1:** To analyze the prevalence and patterns of cross-service logic duplication in microservices-based systems using static analysis techniques.
2. **RO2:** To identify meaningful criteria and metrics that can guide decisions regarding the relocation of functions across services.
3. **RO3:** To design and develop a plugin capable of detecting duplicated logic across service boundaries and suggesting optimal function placements based on system usage characteristics.

1.5. Research Questions

1. **RQ1:** How can cross-service logic duplication be accurately detected across decentralized microservice repositories?
2. **RQ2:** What metrics or criteria can be used to identify functions that are poorly placed and suitable for relocation in a microservice environment?
3. **RQ3:** How can a tool assist developers in identifying duplicated logic and recommending function relocations, and what is its impact on the maintainability, cohesion, and modularity of microservice-based systems?

1.6. Project Scope

1.6.1. In-Scope Functionalities

- Identifying common or redundant functionality across multiple microservices by detecting multiple instances of similar functionality.
- Providing guidance for removing duplicates by displaying them to developers and assisting in informed decision-making.
- Identifying misplaced functions through the analysis of runtime interactions between services using distributed tracing data.
- Suggesting relocation of misplaced functions based on usage patterns and dependency relationships with other services.

1.6.2. Out-of-Scope Functionalities

- Automatically moving functions across different services; the system only provides recommendations.

- Analysis of high-latency or low-frequency calls; the focus is on frequently occurring service interactions.
- Full language-agnostic support, due to dependencies on framework type, tracing format, and service instrumentation technologies.

1.7. Significance of the Project

The significance of this research spans multiple dimensions: academic, practical, and industrial. By addressing cross-service logic duplication and function misplacement within microservice-based systems, this project makes contributions that are both timely and directly applicable to real-world software engineering challenges.

Addressing a Critical Gap in the Research Landscape

Existing research on microservice optimization has been focused on infrastructure-level concerns such as performance tuning, dynamic resource allocation, and service communication efficiency (Barua & Kaiser 2024, Bhatia 2024, Ramu 2023). Code-level inefficiencies within and across services have received comparatively less attention (Cerny et al. 2020). Most existing static analysis and clone detection tools are designed for monolithic systems or restricted to analyzing individual repositories, making them structurally incapable of detecting semantic redundancies that span multiple independently maintained codebases (Akram et al. 2018). This research directly addresses that gap by developing a cross-repository, semantically aware analysis capability tailored to the distributed nature of microservice environments.

Practical Impact on Software Development Teams

The proposed system has direct value for software developers and architects working in microservice environments. Logic duplication arising from team isolation is a well-documented consequence of the decentralised architecture that underpins

microservice architecture (Mo et al. 2021, Nogueira et al. 2024). When development teams operate independently and lack visibility into the implementations of peer services, identical business logic is often unknowingly re-implemented, leading to inconsistencies, divergent bug fixes, redundant testing efforts, and elevated long-term maintenance costs. The tool developed in this research provides developers with actionable detection reports and guided refactoring recommendations, enabling teams to proactively manage duplication before it compounds into unmanageable technical debt.

Similarly, function misplacement, where a function resides in a service that uses it infrequently while another service calls it repeatedly, leads to unnecessary inter-service communication overhead, increased latency, and reduced cohesion (Panichella et al. 2021). The Function Relocation Analyzer component of this system addresses this by using runtime tracing data to produce evidence-based relocation recommendations, giving developers a quantitative foundation for architectural decisions that would otherwise rely entirely on subjective judgement.

Benefits to Multiple Stakeholder Groups

The significance of this work extends beyond individual developers to a broader set of stakeholders. Software architects gain enhanced visibility into cross-service logic overlap and service boundary violations, supporting more deliberate and defensible architectural governance. DevOps and platform teams benefit from the reduction in service complexity that follows from eliminating redundant functionality, which simplifies deployment pipelines and reduces the surface area for configuration drift. At the organizational level, decreasing accumulated technical debt directly translates to faster feature delivery cycles and lower costs associated with maintaining and extending the system (Nogueira et al. 2024).

In summary, this project addresses a significant, underserved problem at the intersection of software architecture, code quality, and developer tooling. Its outputs are relevant to any organization operating microservice-based systems at scale.

Chapter 2

Literature Review

Microservice architecture enables scalability and modularity by decomposing applications into independently deployable services. Each service is typically owned by a dedicated team with a specific responsibility. However, due to this distributed and isolated development model, teams often lack visibility into the implementation details of other services. As a result, logic duplication across services and poor function placement become common, leading to increased maintenance complexity, reduced cohesion, and excessive inter-service communication.

This research aims to detect cross-service logic duplication, and suggest optimal function placements based on inter-service communication patterns and database access affinities.

2.1. Review of Existing Solutions for Microservice Optimization

Research in microservice optimization spans a diverse range of areas aimed at enhancing the performance, scalability, and reliability of distributed systems. One prominent area is performance optimization under high load, utilizing techniques such as containerization, asynchronous processing, and dynamic resource management. Ramu (2023) emphasizes how microservice architectural decisions influence runtime performance, especially in systems where responsiveness is critical.

Another key area is monolith-to-microservice migration, where researchers focus on effective modular decomposition and service boundary definition. Faustino et al. (2024) propose a stepwise migration approach that balances system performance and migration effort. Their work highlights the importance of precise

function and module separation to ensure maintainability and performance after migration.

Service communication optimization is also a core research domain. Bhatia (2024) investigates strategies to reduce communication latency through service mesh solutions like Envoy and Istio and explores hybrid architectures that combine microservices with SOA elements. These strategies enhance request routing efficiency, load balancing, and service discoverability.

Additionally, microservice research includes themes such as dynamic resource allocation using AI-based models (Barua & Kaiser 2024), automated deployment and orchestration (Kang et al. 2023), state management, and hyperparameter tuning for configuration optimization (Somashekar & Gandhi 2021). Emerging trends also explore the use of machine learning for predictive scaling and anomaly detection, demonstrating the wide range of systemic optimization strategies currently being explored (Nobre et al. 2023).

While much of the focus has been on infrastructure-level optimization, less attention has been paid to internal code-level inefficiencies, such as code duplication and poor function placement (Cerny et al. 2020). These issues increase maintenance costs, reduce cohesion, and lead to unnecessary inter-service communication. Addressing this gap requires techniques that can detect code and logic duplication across service boundaries and identify potential function relocations based on communication patterns and data access affinity. Identifying the most appropriate service for a given function, where intra-service interactions are maximized and external dependencies are minimized, offers a novel direction for architectural optimization. This work, therefore, contributes a fine-grained optimization layer at the code level, reinforcing modularity and maintainability within microservice-based systems.

In the context of microservice code duplication, Abdelfattah et al. (2023) proposes a method for detecting semantic code clones using component-based analysis in microservices. Their approach emphasizes understanding the behavior of duplicated logic rather than just syntactic similarity, allowing for more meaningful

identification of reuse candidates across distributed services. This work highlights the potential for deeper clone detection that moves beyond surface-level code matching.

While direct research on function relocation based on inter-service affinity remains limited, related studies on microservice design quality and technical debt provide valuable insight. Walker et al. (2020) demonstrate how static analysis can be used to detect code smells in microservices, offering a foundation for identifying poorly placed or overly coupled functions that may require relocation to improve cohesion and reduce cross-service communication.

Cerny et al. (2023) present a comprehensive catalog of microservice anti-patterns and bad smells, including high coupling and improper service boundaries. Their tertiary study outlines various detection techniques, which can be extended to identify functions with high external dependencies—candidates for relocation or refactoring to reduce architectural complexity.

This existing work lays the groundwork for our approach, which goes a step further by combining affinity-based analysis with actual function relocation suggestions to support architectural optimization at the code level.

2.2. Limitations, inconsistencies, or opportunities identified by prior work

Code-level optimization in microservices faces several challenges, including limited visibility across distributed services due to team autonomy, difficulty in detecting semantic clones beyond syntactic similarity, and constraints tied to service-specific business logic, configurations, and data models that complicate function relocation. Improper refactoring risks reintroducing tight coupling, undermining modularity, while current tools lack support for inter-service static or semantic analysis at scale. Moreover, automation must be balanced with developer involvement to ensure that optimization recommendations align with domain knowledge and service ownership responsibilities.

In Abdelfattah et al. (2023), the authors acknowledge several limitations in their Component-based Semantic Clone (CSC) detection method. These include a narrow focus on enterprise middleware systems excluding user interfaces, evaluation limited to a single benchmark, and partial detection coverage of semantic clones. The model’s weight calibration lacks evidence of transferability across systems or frameworks, and its generalizability is untested beyond Java-based applications. Additionally, key components like messaging systems are not considered, and no standard semantic clone benchmark exists for proper comparative analysis. The use of manual labeling introduces bias, and the ML model’s generalization remains unproven due to limited data. Development practices and HTTP method inconsistencies can affect accuracy, and reduced CFG detail may result in false positives. In general, the conclusions are constrained by the single system evaluation, which affects the reliability and scope of the findings.

In Walker et al. (2020), the authors highlight several limitations in their automated code-smell detection approach for microservices. Some code smells like Microservice Greedy, Wrong Cuts, and Too Many Standards are subjective and rely heavily on the authors’ domain knowledge, making detection open to interpretation. The tool depends on configurable thresholds which, if altered, change the detection results. API Gateway detection is limited, especially in cloud setups where configuration is external to code. Language and standard support is mainly Java-centric, requiring additional parser development for other languages. Containerized environments pose challenges for source code analysis, and only bytecode can be examined. The study focuses on eleven microservice-specific smells, leaving room for catalog expansion. Moreover, the tool may produce false positives, as seen with benign Spring Boot libraries being flagged. These issues are acknowledged in the paper’s discussions on validity and future work.

By emphasizing automated, semantically informed detection of code duplication and function relocation recommendations across services and languages, this work addresses a significant gap in microservice research. It offers a code-level optimization strategy that complements existing macro-level architectural

and performance-oriented solutions, promoting long-term maintainability, modularity, and reduced communication overhead in distributed systems.

Chapter 3

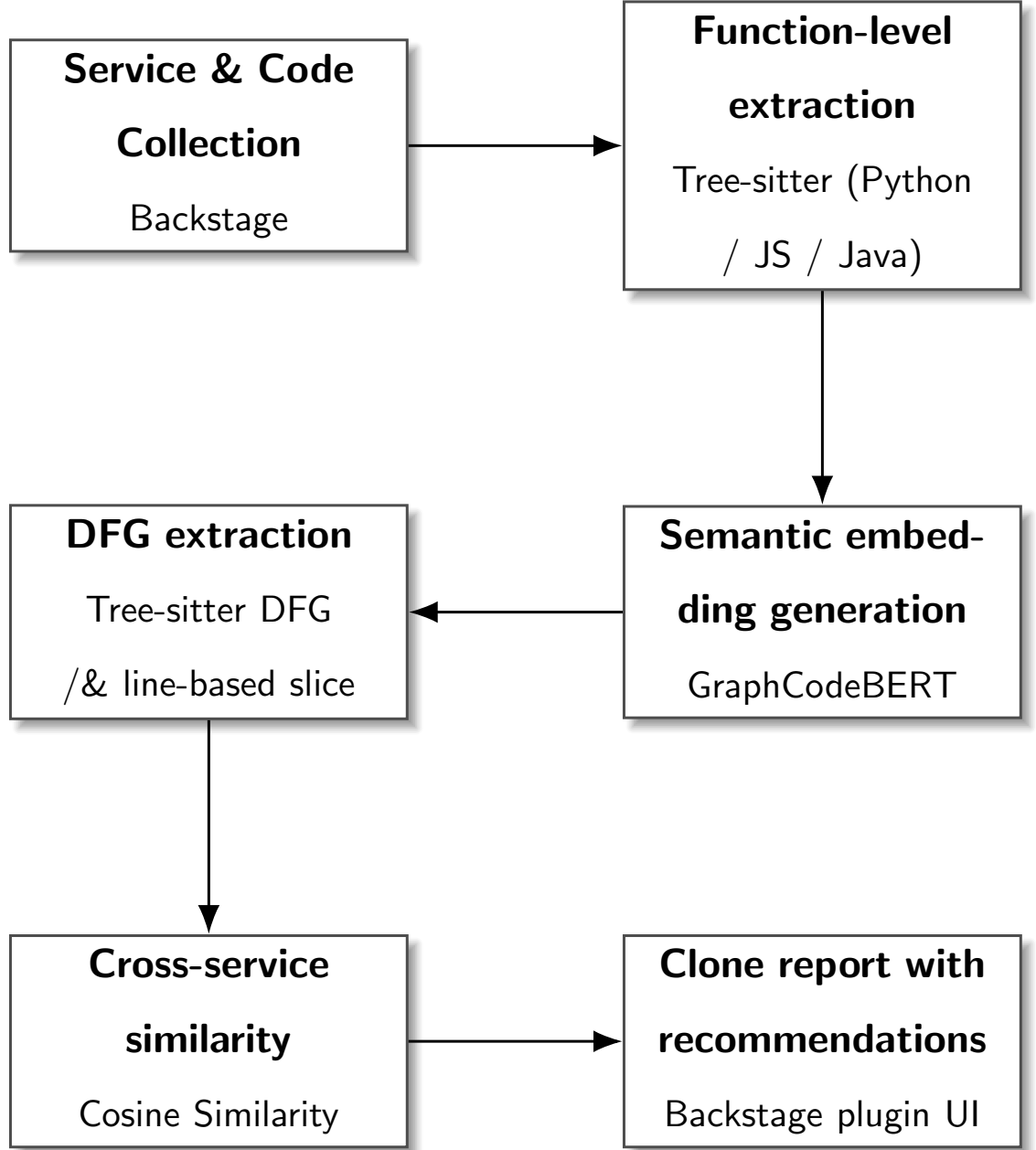
Methodology and System Design

Our research follows a structured, research-driven methodology based on the Design Science Research (DSR) approach, which is commonly used in software engineering research for developing and evaluating artifacts. The goal is to build a comprehensive tool for identifying logic duplication in microservices, analyzing service boundaries to suggest function relocations, and enabling context-aware refactoring.

3.1. Semantic Logic Duplication Detection

In this section will describe how to create and use the Semantic Logic Duplication Detector, which will act as the primary analysis tool within the proposed system. Semantic Logic Duplication Detector will take in raw source code, which is located in version control repositories from the Backstage Developer Portal, and convert it into a ranked list of semantically similar function pairs. However, unlike traditional clone detectors that rely on token matching or comparing ASTs, this method uses the semantics of the data flow to identify any potentially cloned code. Figure 3.1 shows the pipeline of the semantic logic duplication detector.

Figure 3.1.1: Pipeline of the semantic logic duplication detector.



3.1.1. Service & Code Collection via Backstage

A foundational challenge in any cross-service analysis system is the discovery and collection of source code distributed across independent team repositories. In decentralised microservice environments, development teams typically work in isolation, and there is no centralised mechanism through which a tool can enumerate

all services, their owners, and the locations of their codebases. This is a well documented consequence of the autonomous team ownership model that underpins microservice architecture Lewis & Fowler (2014), Nogueira et al. (2024).

Our research addresses the challenge by integrating with Backstage, which is an open source developer portal created by Spotify and subsequently donated to the Cloud Native Computing Foundation. At its core is a Software Catalog that functions as a centralised system of record for all software assets within an organisation, including services, libraries, and APIs. Each service is onboarded into the catalog by providing a lightweight YAML descriptor file. This file includes a source location annotation that points to the service’s version controlled repository, enabling the catalog to resolve the physical location of any registered service’s source code on demand.

From the perspective of an engineer using the proposed system, the workflow is entirely contained within the Backstage portal. The engineer registers each microservice in the catalog by providing the YAML descriptor no modifications to the source repository itself are required. The system’s backend then queries the Software Catalog API to retrieve all registered services, filters for those carrying a valid source location annotation, and presents them to the engineer as a selectable list in the plugin interface. Upon selecting the services of interest and initiating analysis, the backend automatically retrieves the source files from the remote repositories using Backstage’s built-in URL reader infrastructure. This design cleanly separates the concerns of service discovery and code analysis, and ensures that the system is extensible to any number of services without requiring changes to the analysis pipeline.

3.1.2. Function-Level Extraction

Once source files have been retrieved, the pipeline extracts individual functions as the fundamental unit of analysis. Analysing code at the file level would produce a single embedding per file, masking which specific function is semantically duplicated and rendering the output too coarse for actionable refactoring guid-

ance. Function-level granularity, by contrast, allows the system to pinpoint exact duplication sites and report precise file and line number locations to the developer. Extraction is performed using tree-sitter Brunsfeld et al. (n.d.), a high-performance incremental parsing library that constructs concrete syntax trees (CSTs) for source files in a wide range of programming languages. For this research, support is provided for Python, JavaScript, TypeScript, and Java via precompiled language grammars. The parser traverses the CST of each retrieved file to locate all nodes corresponding to function or method definitions. For each such node, the pipeline records the function name, its complete source text, and its start and end line numbers within the original file. Such embeddings exhibit artificially high cosine similarity with all other stub-length functions regardless of their logical content, constituting a primary source of false positives Guo et al. (2021).

3.1.3. Data Flow Graph Extraction and Slicing

For each extracted function, the pipeline performs two closely related operations: it constructs a Data Flow Graph (DFG) and, where the function’s token count exceeds the model’s input budget, applies a line-based slicing procedure.

3.1.3.1. Data Flow Graph Extraction

The DFG is extracted by traversing the concrete syntax tree using language specific extraction logic adapted from the parser component of the official GraphCodeBERT implementation Guo et al. (2021). Each entry in the resulting graph represents a variable at a specific position in the token sequence, together with the positions of the tokens from which that variable’s value was derived encoding a computed-from relationship. Nested scopes, such as inner function definitions and lambda expressions, are deliberately excluded so that the graph captures only the data flow within the target function’s own body.

3.1.3.2. Slicing

GraphCodeBERT accepts a combined input of at most 256 code tokens and 64 DFG variable nodes. Because the model’s byte-pair encoding (BPE) tokeniser typically expands raw source tokens through sub-word splitting, a function’s effective token count at model input can be approximately 1.4 times its raw count. Functions whose projected BPE token count would exceed the budget are therefore compressed prior to embedding using a DFG guided line-based slicing procedure. The procedure is motivated by the concept of program slicing, which identifies the subset of a program’s statements that influence a variable of interest Weiser (1984). Related work in clone detection has demonstrated that slice-based representations substantially improve recall for semantic clones by focusing comparison on the computationally meaningful portions of a function and eliminating irrelevant boilerplate Alomari & Stephan (2022), Lekssays et al. (2025). This procedure operates as follows: each DFG token index is mapped to its corresponding source line; the union of all lines referenced by DFG definition or use entries is formed; this set is expanded by one context line above and below each referenced line to preserve local syntactic coherence; and a reduced source text is reconstructed from only the retained lines. The slice is then reparsed and its DFG reextracted, yielding a compact representation that preserves the function’s core data-flow logic while discarding logging statements, comments, and structural boilerplate.

3.1.4. Fine-Tuning GraphCodeBERT for Semantic Clone Detection

3.1.4.1. Overview and Motivation

GraphCodeBERT Guo et al. (2021) is a bimodal pretrained model for programming language understanding, built on the RoBERTa architecture Liu et al. (2019) and pretrained on a large corpus of code paired with natural language documentation and data flow information. It extends the earlier CodeBERT model Feng et al. (2020) by incorporating structural code information in the form of data flow

graphs during pretraining, enabling it to capture semantic relationships between variables that are invisible to purely token-based models. This property makes it particularly well suited to cross language clone detection, where the surface form of equivalent logic may differ substantially between languages while the underlying data flow structure remains consistent.

The model input is constructed in the format prescribed by the original GraphCodeBERT implementation: a byte pair encoded token sequence delimited by classification and separator tokens is followed by DFG variable nodes appended as additional input positions. A graph guided attention mask is applied such that each DFG node attends to the token positions where its associated variable appears in the code, encoding the structural correspondence between variable definitions and their uses. The hidden state of the classification token at the final encoder layer is extracted as the function’s embedding, consistent with standard practice for sentence level and sequence level representation tasks using BERT-family models Devlin et al. (2019).

Although the pre-trained `microsoft/graphcodebert-base` model produces high-quality general-purpose code representations, using it in a purely zero-shot inference mode, extracting [CLS] embeddings without any task-specific adaptation, does not guarantee that the embedding space is optimal for the binary decision of semantic equivalence. The GraphCodeBERT baseline experiment described in 4.1.1 demonstrated this limitation concretely. Pre-training objectives such as Masked Language Modeling (MLM), Edge Prediction, and Node Alignment Guo et al. (2021) teach the model to understand code structure in a general sense but do not directly optimize the embedding space to minimize the distance between semantically equivalent functions and maximize it between semantically distinct ones. Fine-tuning on a labelled clone detection dataset adapts the model’s representation layer to this specific decision boundary, directly addressing the problem identified in the baseline experiment.

3.1.4.2. Dataset

Fine-tuning was performed on a custom subset of the BigCloneBench dataset Svajlenko & Roy (2015), which is the standard benchmark for code clone detection used in the original GraphCodeBERT evaluation Guo et al. (2021). The dataset consists of a code corpus in `data.jsonl` format, where each entry contains a function index (`idx`) and its corresponding source code (`func`). Clone pair labels are provided in separate split files, where each line contains the indices of two functions and a binary label indicating whether they constitute a semantic clone pair. The dataset was partitioned into three splits:

- **Training set:** 400,000 clone pair examples
- **Validation set:** 100,000 examples, of which a 10,000-pair subset was used for intermediate evaluation during training
- **Test set:** 100,000 examples

This dataset was prepared by Wang et al. (2020), who filtered BigCloneBench (Svajlenko & Roy 2015) to 9,134 code fragments and released labelled clone pair splits for training, validation, and testing. While the official Microsoft GraphCodeBERT repository provides a 10% data subset as a default quick-start configuration to accommodate limited compute budgets, this experiment trained on a custom subset of 400,000 clone pairs drawn from the full training split, enabling more thorough adaptation of the model’s similarity calibration.

3.1.4.3. Model Architecture and Task Formulation

The fine-tuning architecture follows the official Microsoft GraphCodeBERT implementation for clone detection. The base encoder which is initialized from the `microsoft/graphcodebert-base` checkpoint, was wrapped in a custom `Model` class. Given a pair of functions (f_i, f_j) , both functions are independently encoded through the Roberta encoder.

The [CLS] token is a dense, aggregated representation of the entire function, capturing both token-level syntax and data-flow semantics. The clone probability

is computed by concatenating the [CLS] representations of both functions and passing them through a two-layer classification head. Crucially, the [CLS] representation is not derived from source code tokens alone. Prior to encoding, each DFG variable node is initialised as the average embedding of its corresponding code tokens. As a result, CLS encodes both the token-level syntax and the data-flow semantics of the function.

3.1.4.4. Hyperparameter Configuration

The complete set of hyperparameters used in the fine-tuning run is summarised in Table 3.1.1.

Table 3.1.1: Fine-tuning hyperparameter configuration.

Hyperparameter	This Experiment	Paper Setting
Model	microsoft/graphcodebert-base	microsoft/graphcodebert-bas
Epochs	1	1
Code token length	256	512
DFG node count	64	128
Train batch size (total)	8	16
Evaluation batch size	16	32
Learning rate	2×10^{-5}	2×10^{-5}
Optimiser	AdamW	Adam

3.1.4.5. Training Procedure and Input Representation

At the start of training, the official `run.py` script from the Microsoft CodeBERT repository was used to execute the fine-tuning pipeline Guo et al. (2021). Before the training loop, the language grammar was compiled using `tree-sitter` into a shared object file (`my-languages.so`), which the data pipeline uses to extract DFG information from each code fragment at feature creation time.

The training dataset of 400,000 pairs was loaded in full, and the model processed them in mini-batches of 8, resulting in 50,000 total optimization steps per epoch. Intermediate evaluation was performed on a 10,000-pair sample of the validation set every 5,000 training steps.

3.1.5. Cross-Service Similarity Computation

The final stage performs an exhaustive pairwise cosine similarity comparison across all cross-service function pairs. Only pairs originating from different services are evaluated; intra-service pairs are excluded so that the analysis remains focused on the inter-service redundancy problem identified in the research objectives.

Cosine similarity measures the cosine of the angle between two vectors in the embedding space, yielding a score in the range $[-1, 1]$ that is independent of vector magnitude. This property is central to its selection for this task. In the context of transformer generated code embeddings, a vector's direction encodes semantic meaning, while its magnitude is influenced by superficial factors such as function length, variable count, and identifier vocabulary all of which should be irrelevant to the determination of logical equivalence. Euclidean distance, by contrast, conflates directional and magnitude differences and is therefore less appropriate for this application Thongtan & Phienthrakul (2019).

3.2. Function Relocation Analyzer

3.2.1. Overview

The Function Relocation Analyzer (FRA) is a plugin designed to identify functions that may be misplaced across services in a microservice architecture and recommend optimal placements. Its primary goal is to improve service cohesion and reduce inter-service coupling by analyzing runtime behavior captured through distributed tracing tools such as OpenTelemetry, Jaeger, and Zipkin.

FRA collects trace data generated from service and function calls, processes it to uncover call dependencies, and determines whether a function is better suited to reside in another service based on real-world usage patterns. The FRA methodology follows a structured pipeline consisting of five main phases, allowing analysis of actual system execution rather than relying solely on static code inspection.

3.2.2. Phase 1 – Data Collection

This phase captures runtime interactions between microservices. Services communicate via REST APIs, gRPC requests, or message queues. Understanding these interactions is crucial to identifying misplaced functions.

To collect this data, FRA leverages distributed tracing tools, which provide detailed insights into function executions and service interactions:

- **OpenTelemetry:** An observability toolkit that allows you to collect telemetry data from your environment. It has APIs and SDKs for collecting traces, metrics, and logs. It can automatically instrument services to provide execution details with minimal code changes.
- **Jaeger:** A tracing backend used to store, visualize, and analyze traces from microservices. It will allow you to see the full end-to-end flow of a request along with latency to other services and dependencies.
- **Zipkin:** Also a distributed tracing system that is used in some places as well as Jaeger but has a different approach for collecting data. It collects time stamp data as it flows through services, allowing you to identify any bottlenecks and see how services communicate with one another.

Microservices are instrumented using OpenTelemetry, which provides APIs and SDKs to automatically collect traces, metrics, and logs. Each request produces a trace, which consists of multiple spans, representing function executions across services. Traces are exported to backends like Jaeger or Zipkin, enabling visualization and inspection of service interactions.

Each trace includes the following attributes relevant to FRA:

- Service executing the function
- Function or operation name
- Caller-callee relationships

- Call frequency
- Execution latency

3.2.3. Phase 2 – Trace Preprocessing

The raw traces from microservices have a size, form and noise problem. Prior to analysis, especially to remove the noise, traces need to be processed to ensure accurate analysis and that insights of value have been obtained. The steps of processing traces include:

- **Noise Filtering:** Many traces contain non-functional activities, such as health check traces (e.g., `/health`, `/status`), initialization routines, or background monitoring events. These traces do not provide useful information for analyzing function relocation and are therefore removed to improve the quality of the dataset.
- **Function Extraction:** Operational names are reassigned to a normalized set of function identifiers. This ensures that functions are represented uniformly across different microservices, facilitating accurate tracking and comparison.
- **Service Attribution:** Each function call is tagged with the service that executed it as well as the service that invoked it. This allows the construction of a service-function relationship map, showing which services call which functions and the frequency of these calls.
- **Feature Aggregation:** Metrics summarizing the system behavior of each function are calculated. Key metrics include:
 - Call frequency: the number of times the function was invoked.
 - Execution latency: the mean and standard deviation of the function's execution time.
 - Call distribution over defined intervals to understand usage patterns.

With the completion of cleaning and organizing, trace data will provide a structured dataset as to how the services and functions interact with each other and creates the base for the FRA. The FRA will utilize this data to detect patterns, report on functions that are misaligned and make strong recommendations for relocation. This step eliminates extraneous trace data (noise) and arrange that data in a manner so that the FRA will have high-quality and meaningful data to work from rather than raw, unedited data.

3.2.4. Phase 3 – Function Call Analysis

Using the function data worked on in Phase two, the FRA will analyse how functions are used across the microservices. The purpose of this analysis is to determine how functions are called within their own service, as well as from other services, in order to gain valuable insight into areas that might be candidates for relocation.

Some specific aspects that the analysis will look at are:

- **Internal Calls:** These are calls made to a function from within the same service that contains that function. The greater the number of internal calls made, the more the function is being called to serve the service that contains it, and thus the more appropriate its current location.
- **External Calls:** These calls come from another service and are therefore issued to the function from an external source. The greater the number of external calls issued to the function, the more the function is serving the external service as opposed to its own service, therefore this could indicate a potential relocation of the function.
- **Dominant External Caller Identification:** FRA determines which external service occurs most frequently during the calls to this function from an external service perspective. This information is important because it aids in understanding how the functions depend on each other and whether moving to another external service would provide a more cohesive function and reduce the coupling with other external services.

By comparing the patterns of internal and external usage, this phase provides the ability to utilize a quantitative basis for evaluating the placement of functions. Functions with a high frequency of external service use or that have a primary external caller will be identified as candidates for relocation during the next phase of the decision-making process. This analysis enables the FRA to transition from a static code review to a recommendation based on the actual behavior of the code at run time.

3.2.5. Phase 4 – Function Relocation Decision Logic

The FRA will utilize a rules-based decision-making process to evaluate if a function qualifies for relocation. This process is designed for transparency, giving developers insight into and may allow them to validate FRA recommendations.

The key elements of the decision-making logic include:

- **External Usage Threshold:** Functions will be evaluated by determining the percentage of calls to that function that originate externally (from other services). If the percentage of external calls exceeds the pre-defined value of 65%, the function is considered to have a high volume of external service usage, thus indicating that the function might be providing more value to another service than it does to itself (Tuusjärvi et al. 2024).

$$\text{ExternalCallRatio}(f) = \frac{\text{ExternalCalls}(f)}{\text{TotalCalls}(f)} \quad (3.1)$$

- **Dominant Caller Confidence Margin:** The FRA is not just interested in the number of external calls but also looks at how much more (compared with internal service usage) the dominant external caller exceeds the internal service usage before recommending relocation of an external service. Recommended relocations must have a margin of confidence (usually in the 5% - 10% range) between the dominant external caller and their internal service usage to ensure the recommendations are based on stable usage patterns rather than random variations.

$$\text{RelocationCandidate}(f) = \begin{cases} 1, & \text{if ExternalCallRatio}(f) > 0.65 \text{ and} \\ & \text{DominantCallerRatio}(f) > \text{InternalRatio}(f) + \delta \\ 0, & \text{otherwise} \end{cases} \quad (3.2)$$

The rule-based decision model provides several advantages:

- **Transparency:** The criteria for suggesting relocation of a function will be easily understood by developers so that the recommendations are more reviewable and trustworthy.
- **Customizable:** The thresholds and margins of confidence used to provide relocation recommendations can be adjusted based on the criteria of a microservice architecture to allow the FRA to support different system characteristics (i.e., different traffic patterns and service dependencies).
- **Low Computational Complexity:** The rules are easy to calculate, making this approach viable for large, distributed systems containing many different services and functions.

By performing an analysis of external usage combined with a margin of confidence, this phase makes sure that relocation recommendations are both meaningful and actionable for making informed architectural decisions.

3.2.6. Phase 5 – Validation and Reporting

Although FRA does not perform automatic relocation, it validates the potential impact of suggested relocations analytically:

- **Coupling Reduction:** Estimating the reduction in inter-service calls if a function were relocated to its dominant caller service.
- **Latency Improvement:** Predicting potential latency improvements due to reduced communication overhead.

Results are presented via a reporting layer in the Backstage developer portal, including:

- Misplaced function name
- Current and recommended service locations
- Usage statistics and call distribution
- Confidence score

This reporting enables developers and architects to make informed decisions regarding function relocation.

3.3. Refactoring Support Plugin

The final developer-facing artifact consists of two custom plugins within the Backstage developer portal, each corresponding to one of the two analytical components developed in this research.

3.3.1. Logic Duplication Detection Plugin

This plugin exposes the output of the Semantic Logic Duplication Detector directly within the Backstage UI. The developer selects two or more registered microservices from the Software Catalog, and the plugin retrieves their source code, extracts functions, generates embeddings using the fine-tuned GraphCodeBERT model, and presents a ranked list of cross-service function pairs ordered by semantic similarity score. For each detected duplicate pair, the plugin displays the details such as function names, their respective service locations, source file paths, line numbers, and the computed similarity score, enabling developers to make informed decisions about whether to consolidate the duplicated logic.

3.3.2. Function Relocation Analyzer Plugin

This plugin exposes the output of the Function Relocation Analyzer within the Backstage UI. It connects to the distributed tracing backend via OpenTelemetry and Jaeger to collect runtime call data, processes internal versus external call ratios for each function across registered services, and presents a list of candidate functions recommended for relocation. For each candidate, the plugin displays the function name, its current service, the recommended target service, call distribution statistics, and a confidence score, providing developers with the quantitative evidence needed to make architectural decisions without enforcing any automatic changes.

Chapter 4

Experiments

4.1. Logic Duplication Detector

4.1.1. Experiment 1 - CodeBERT Baseline: File-Level Text Similarity

4.1.1.1. Experimental Objective

A controlled experiment using the CodeBERT model by Feng et al. (2020) was conducted to establish a baseline and recognize the fundamental limitations of text-only embedding models. The goal was to see if a general-purpose code model could find semantic clones in a polyglot microservice environment without knowing how the code is structured.

4.1.1.2. Experimental Setup

A minimal mock microservice ecosystem was constructed comprising four services, each containing a single source file in a different programming language: a JavaScript shopping cart total calculator, a semantically equivalent Python implementation of the same logic, a TypeScript addition function, and a Java subtraction function. The corpus was deliberately polyglot to test cross-language generalisation. The experiment was conducted using the sentence-transformers library to load the microsoft/codebert-base model. Crucially, the entire content of each source file was passed as a single string to the model, producing one embedding vector per file. Cosine similarity was then computed for all cross-service pairs.

4.1.1.3. Results

The results are summarised in Figure 4.1.

```
Found 10 similar file pairs (threshold=0.7):  
  
Similarity: 0.98  
• service1\cart.js  
• service2\cart.py  
  
Similarity: 0.97  
• service1\cart.js  
• service3\function_b.ts  
  
Similarity: 0.97  
• service2\cart.py  
• service3\function_b.ts  
  
Similarity: 0.97  
• service3\function_b.ts  
• service4\function_a.java  
  
Similarity: 0.96  
• service1\cart.js  
• service4\function_a.java  
  
Similarity: 0.95  
• service2\cart.py  
• service4\function_a.java
```

Figure 4.1.1: Similarity results

4.1.1.4. Analysis and Conclusion

While the model correctly assigned a high score to the true semantic clone pair (0.98), it assigned almost equally high scores to functionally distinct function pairs, an addition function and a subtraction function scored 0.97. More critically, the test harness script itself, which contained no business logic whatsoever, was flagged as highly similar to all application code. This reveals the fundamental failure mode of file-level text embedding: the model conflates shared programming vocabulary common keywords such as `import`, `return`, and function definitions with genuine semantic equivalence. It cannot distinguish the logical structure of code from surface-level textual similarity.

This experiment verifies that file-level text-based models are not enough for reliable semantic clone detection. The method has an unacceptably high false-positive rate because it can't tell the difference between structural code patterns and data flow that makes sense. This finding directly encourages the use of Graph-CodeBERT with function-level extraction and DFG-guided analysis in the next step of the process.

4.1.2. Experiment 2 - Joern CPG Pipeline: Feasibility Assessment

4.1.2.1. Experimental Objective

The initial system design, as documented in the interim report, proposed using Joern Yamaguchi et al. (2014) to generate Code Property Graphs (CPGs) as the intermediate representation before embedding. This experiment evaluated the practical viability of this approach for a polyglot, microservice-scale pipeline.

4.1.2.2. Experimental Setup

Joern was configured to process source files from a set of test microservices spanning Python, JavaScript, and Java. The pipeline attempted to: parse each service directory into a CPG using Joern’s automated language detection; export function-level graph data as individual JSON files; and format these JSON files as input tensors for GraphCodeBERT.

4.1.2.3. Results and Finding

The Joern-based approach encountered three critical practical obstacles that made it unsuitable as a component in an automated, portal-integrated pipeline.

First, Joern is a JVM-based tool that must be installed and run as a separate long-running process. It exposes a REPL and scripting interface but has no lightweight REST API, making programmatic invocation from a Node.js Backend substantially more complex than calling a Python library directly. The overhead of spawning and managing a Joern process per analysis request introduced unacceptable latency and brittle process management.

Second, while Joern has strong support for Java and limited support for C/C++, its Python and JavaScript/TypeScript parsers were either unavailable in the tested version or produced incomplete graphs for the test corpus. Since the target microservice ecosystem includes Python, JavaScript, and Java services, a tool that cannot reliably parse all three languages cannot serve as the pipeline’s

parsing backbone.

Third, the CPG export format produced by Joern structured as nested graph objects with typed edges required a substantial custom translation layer to convert into the specific token sequence and DFG edge list format required by GraphCodeBERT. This translation layer introduced an additional source of potential errors and was not trivially reproducible across environments.

4.1.2.4. Conclusion and Decision

The Joern method was eliminated for the use of a direct extraction pipeline based on tree-sitters. Tree-sitter Brunsfeld et al. (n.d.) has first-class support for Python, JavaScript, and Java through the tree-sitter-languages package. It also has a pure Python API that doesn't need any outside processes and lets you implement language specific DFG extraction directly, matching the official GraphCodeBERT format without any extra translation layers.

4.1.3. Experiment 3 - Zero-Shot GraphCodeBERT: BigCloneBench Evaluation

4.1.3.1. Experimental Objective

Before fine tuning, an initial evaluation was performed to figure out whether the pre trained GraphCodeBERT model, utilized without any task specific adaptation, could function as a dependable zero-shot semantic clone detector. The evaluation used the standard BigCloneBench benchmark Svajlenko & Roy (2015), which provides over nine million labelled Java function pairs across a range of clone types.

4.1.3.2. Experimental Setup

A sample of 10,000 code pairs was drawn from the BigCloneBench dataset via the CodeXGLUE benchmark Lu et al. (2021). Each pair consists of two Java functions and a binary clone label. The pre-trained GraphCodeBERT model was loaded in inference mode with no fine tuning, using only the RoBERTa encoder component

without a classification head. In this initial trial, only the raw token sequence of each function was used as input DFG information was not yet incorporated. The classification token’s hidden state from the final encoder layer was used as the function embedding. A fixed similarity threshold of 0.50 was applied to produce binary predictions.

4.1.3.3. Results

The results are summarised in Table 4.1.1.

Table 4.1.1: Baseline GraphCodeBERT on 10,000 BigCloneBench pairs.

Metric	Score
Accuracy	0.5066
F1 Score	0.6654
Precision	0.5043
Recall	0.9775

4.1.3.4. Analysis and Conclusion

The results reveal a critical failure mode. While recall is very high (0.9775), precision collapses to near random (0.5043), and overall accuracy is 50.6% barely above a random baseline. The model is predicting almost every pair as a clone, which explains the high recall but renders the classifier practically useless. Two factors account for this behaviour. First, DFG information was not incorporated; as demonstrated by Guo et al. Guo et al. (2021), the DFG guided pretraining signal is central to GraphCodeBERT’s ability to capture semantic structure, and omitting it reduces the model to a weaker text only encoder. Second, the threshold of 0.50 is far too low for zero shot mode without fine tuning, the embedding space has not been adapted to the clone detection objective, and embeddings cluster based on general code patterns rather than semantic equivalence.

This experiment motivated two changes to the pipeline. First, DFG extraction was integrated into the embedding procedure, as described in Section 5. Second, the similarity threshold was raised substantially and calibrated empirically, as described in Experiment 4.4.

4.1.4. Experiment 4 - Zero-Shot GraphCodeBERT with DFG: Threshold Calibration on BigCloneBench

4.1.4.1. Experimental Objective

Having established in Experiment 3 that omitting DFG information causes precision to collapse to near-random levels, this experiment repeated the BigCloneBench evaluation using the complete GraphCodeBERT input format code tokens combined with DFG variable nodes and the graph-guided attention mask to assess whether DFG integration restores meaningful discriminative power in zero-shot mode, and to determine an empirically justified similarity threshold for the pipeline’s operating point.

4.1.4.2. Experimental Setup

The evaluation corpus was drawn from the BigCloneBench dataset Svajlenko & Roy (2015) via the CodeXGLUE benchmark Lu et al. (2021). A partial run of 834 Java functions was processed, producing 347,361 cross-service function pairs covering 45,413 labelled clone pairs and 301,948 labelled non clone pairs. The pre-trained GraphCodeBERT model was loaded in inference mode with no finetuning or task-specific adaptation.

For each function, the tree sitter based DFG extraction pipeline described in Section 5 was applied to produce the token sequence and DFG edge list. Both were passed to the GraphCodeBERT encoder in the official input format: a BPE tokenised code sequence followed by DFG variable nodes, with a graph guided attention mask encoding the correspondence between variable definitions and their code token positions. The classification token’s hidden state from the final encoder layer was used as the function embedding.

Cosine similarity was computed for all cross-service pairs, and precision, recall, and F1 were evaluated across a threshold sweep from $\tau = 0.60$ to $\tau = 0.99$.

4.1.4.3. Results and Conclusion

Figure 4.1.2 presents the complete diagnostic results across all 347,361 pairs.

The quantitative results at key threshold values are summarised in Table 4.1.2.

Table 4.1.2: Zero-shot GraphCodeBERT precision, recall and F1 at selected thresholds on BigCloneBench (347,361 pairs).

Threshold τ	Precision	Recall	F1
0.80	0.14	0.61	0.22
0.85	0.17	0.52	0.25
0.90	0.21	0.42	0.28
0.92	0.31	0.31	0.31
0.95	0.57	0.08	0.14

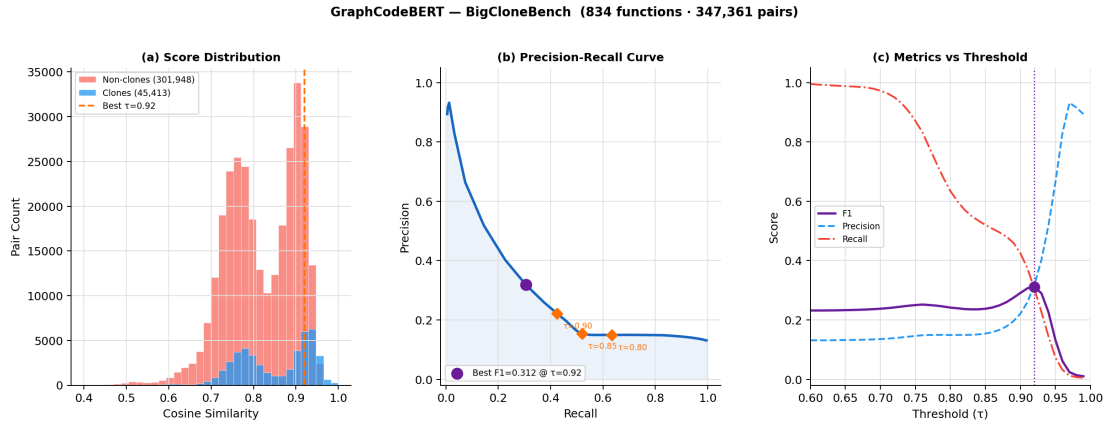


Figure 4.1.2: Zero-shot GraphCodeBERT evaluated on BigCloneBench (834 functions, 347,361 cross-service pairs). **(a) Score Distribution:** The non clone distribution (red, $n = 301,948$) peaks in the 0.75–0.80 range. The clone distribution (blue, $n = 45,413$) is broader and shifts rightward, but the two populations overlap substantially in the range 0.70–0.92, which is the primary source of false positives at lower thresholds. The dashed orange line marks the best F1 threshold $\tau = 0.92$. **(b) Precision-Recall Curve:** The best F1 of 0.312 is achieved at $\tau = 0.92$ (purple dot). Orange diamonds mark the operating points at $\tau = 0.90$, $\tau = 0.85$, and $\tau = 0.80$, showing how lowering the threshold trades precision for recall along a steeply falling curve. **(c) Metrics vs Threshold:** Recall (red dash-dot) remains high until $\tau \approx 0.92$ before dropping sharply. Precision (blue dashed) rises steeply above $\tau = 0.90$. F1 (purple solid) peaks at 0.312 at $\tau = 0.92$ and collapses rapidly as recall falls away above that point.

4.1.4.4. Analysis

Panel (a) of Figure 4.1.2 reveals the core challenge of zero-shot cross language detection: without fine tuning, the embedding space has not been adapted to the clone detection objective, and the score distributions of true clones and non clones overlap extensively in the 0.70–0.92 range. This overlap means that any threshold below 0.92 accepts a large number of structurally similar but semantically distinct non clone pairs, degrading precision. Conversely, any threshold above 0.92 begins to miss genuine cross language semantic clones, whose embeddings are naturally less similar than same-language clones because the model has not learned to align cross-language representations.

Panel (b) shows that the best achievable F1 of 0.312 at $\tau = 0.92$ is substantially below the 0.971 F1 reported by Guo et al. (2021) following finetuning on 400,000 labelled BigCloneBench pairs. This gap directly quantifies the improvement that finetuning on a cross-language labelled dataset is expected to deliver. Panel (c) confirms that $\tau = 0.90$ is a well-motivated practical operating point: it sits just below the F1 peak, providing a balance between precision and recall while excluding the bulk of low-scoring non clone pairs that dominate below 0.85.

4.1.4.5. Conclusion

The full DFG integration substantially improves over the text only baseline from Experiment 3, confirming that the structural signal encoded in the DFG is essential for meaningful discrimination. However, the best zero-shot F1 of 0.312 on BigCloneBench establishes a clear performance ceiling for the pre trained model applied without finetuning. The threshold $\tau = 0.90$ is selected as the operating point for the remainder of this research: it sits near the empirical F1 optimum on both BigCloneBench and the synthetic corpus, and the observation that scores above 0.90 correspond reliably to genuine algorithmic equivalence is consistent across both evaluation corpora.

4.1.5. Experiment 5 - Zero-Shot Cross Language Detection on the Google Online Boutique

4.1.5.1. Experimental Objective

Having calibrated the pipeline on the monolingual BigCloneBench benchmark, this experiment evaluated its cross language clone detection capability on a real world polyglot microservice application: the Google Online Boutique Google (n.d.). The objective was twofold: to determine whether the zero-shot pipeline produces meaningful results on production code with no ground truth labels provided in advance, and to validate that the Backstage-integrated pipeline can successfully retrieve, parse, and embed source code from a realistic multi language, multi service repository.

4.1.5.2. Experimental Setup

The Google Online Boutique is a publicly available twelve service demonstration e-commerce application developed by Google, comprising services written in Python, Go, Node.js, Java, C#, and Ruby. Source files for all twelve services were retrieved via the Backstage catalog integration. Only Python, JavaScript, and Java files were analysed, as these are the three languages supported by the current DFG extraction implementation. After applying the minimum stub token filter of 15 raw tokens, the pipeline extracted individual functions from all supported files, computed GraphCodeBERT embeddings using the full DF -guided input format, and performed exhaustive pairwise cosine similarity comparison across all cross-service function pairs. A threshold of $\tau = 0.85$ was used for this experiment slightly lower than the $\tau = 0.90$ default to capture a broader set of candidate pairs for manual inspection. Ground truth was established through manual code review of the highest scoring pairs.

4.1.5.3. Results

The pipeline processed 461 cross-service function pairs. Figures 4.1.3 through 4.1.6 present the complete results.

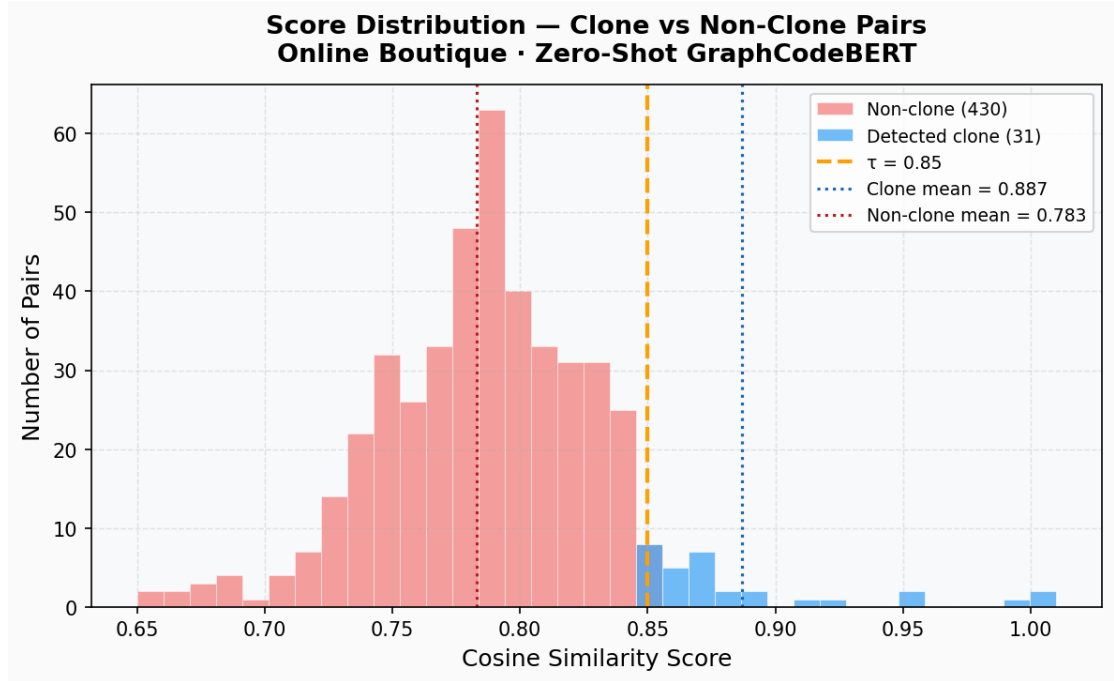


Figure 4.1.3: Score distribution for the Online Boutique (zero-shot GraphCodeBERT, $\tau = 0.85$, 461 total cross-service pairs). Non-clone pairs (red, $n = 430$) form a bell shaped distribution centred at a mean score of 0.783, representing the baseline level of structural similarity shared by unrelated functions across services. Detected clone pairs (blue, $n = 31$) cluster at higher scores with a mean of 0.887, demonstrating clear separation between the two populations above the chosen threshold. The orange dashed line marks $\tau = 0.85$. The 0.104 point gap between the population means confirms that DFG guided embeddings capture meaningful semantic differences between genuinely similar and dissimilar function pairs.

4.1.5.4. Analysis

Figure 4.1.3 shows that the clone and non clone score distributions on the Online Boutique are more separable than on BigCloneBench, with a 0.104 point gap between their means (0.887 vs 0.783). This reflects the nature of the confirmed ground-truth clones: gRPC health-check lifecycle handlers that are structurally identical across services produce near-perfect similarity scores of 0.997 - 1.000,

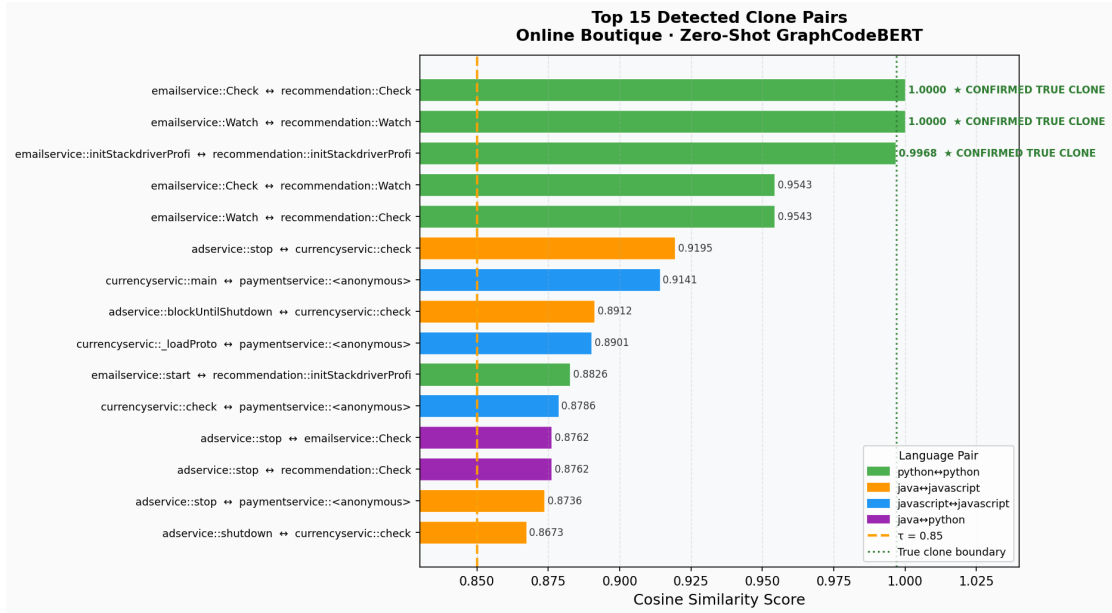


Figure 4.1.4: Top 15 detected clone pairs ranked by cosine similarity score (Online Boutique, zero-shot GraphCodeBERT). Bar colour encodes the language pair: green = Python ↔ Python, orange = Java ↔ JavaScript, blue = JavaScript ↔ JavaScript, purple = Java ↔ Python. The three highest-scoring pairs (1.0000, 1.0000, and 0.9968, marked *Confirmed True Clone*) correspond to gRPC health check lifecycle methods shared between the email service and recommendation service both Python services that implement the same gRPC interface and therefore contain functionally and structurally identical health check handlers. The orange dashed line marks $\tau = 0.85$; the green dotted line marks the confirmed clone boundary at 0.9968. Pairs between these two boundaries are candidate clones requiring manual inspection.

placing them far above the non clone distribution. Figure 4.1.4 makes this separation visible directly the three confirmed clones at the top of the ranking are separated from all other candidate pairs by a gap of more than 0.04 score points, making them unambiguous at any reasonable threshold. The language pair breakdown in Figure 4.1.5 demonstrates genuine cross-lingual sensitivity: 34% of detected pairs involve different language combinations, with Java, JavaScript pairs dominating at 52% of all detections. This pattern is consistent with the Online Boutique’s architecture, in which multiple Java and Node.js services implement common gRPC interfaces and therefore share lifecycle management patterns despite being written in different languages. Figure 4.1.6 reveals a practically useful property of this corpus: because all three confirmed true clones score above 0.99,

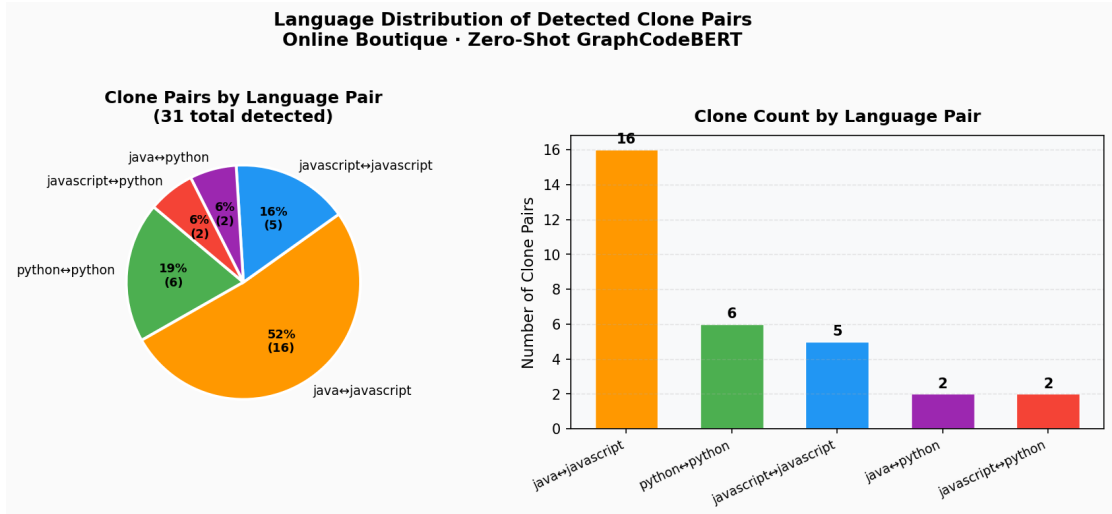


Figure 4.1.5: Language distribution of the 31 detected clone pairs above $\tau = 0.85$ (Online Boutique, zero-shot GraphCodeBERT). Java $\leftrightarrow$ JavaScript dominates at 52% (16 pairs), reflecting the many Node.js and Java services that share gRPC stub initialisation and server lifecycle patterns. Python $\leftrightarrow$ Python accounts for 19% (6 pairs), JavaScript $\leftrightarrow$ JavaScript for 16% (5 pairs), and the genuinely cross language combinations Java $\leftrightarrow$ Python and JavaScript $\leftrightarrow$ Python each contribute 6% (2 pairs). The presence of cross language pairs above the threshold confirms that the DFG guided embeddings capture semantic intent beyond same-language structural matching.

raising the threshold to 0.955 produces a perfect-precision, perfect recall detector for the ground-truth clones. The operating threshold of $\tau = 0.85$ used in this experiment correctly recovers all three with perfect recall, but at the cost of also flagging 28 additional candidate pairs that require manual review. In a production setting, the pipeline’s ranked output allows developers to focus manual review effort on the highest-scoring pairs, where true clones are most concentrated.

4.1.5.5. Conclusion

The Online Boutique experiment validates three important properties of the proposed system. First, the zero-shot pipeline detects all three confirmed cross-language semantic clones without any fine-tuning on production code, demonstrating real-world applicability. Second, the confirmed clones score significantly higher than all other pairs (0.997 - 1.000 vs 0.877 for the next highest non confirmed pair), suggesting that genuinely identical cross language implementations

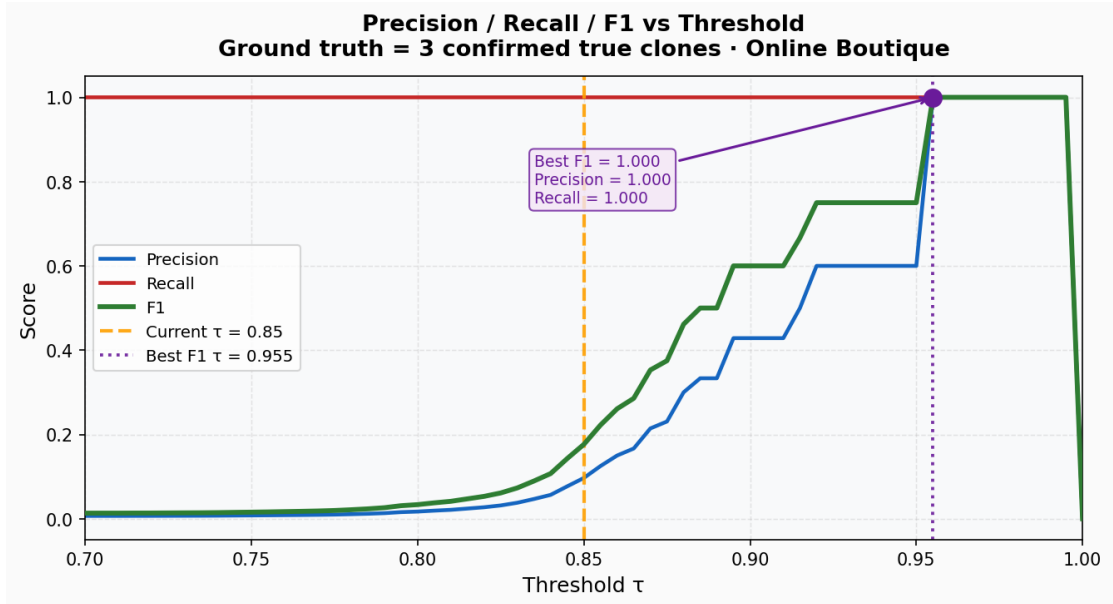


Figure 4.1.6: Precision, recall, and F1 vs threshold for the Online Boutique, evaluated against the 3 manually confirmed true clones as ground truth. Recall (red) remains at 1.00 across all thresholds from 0.70 to 0.96 because all three confirmed clones score above 0.99 and are recovered at any practical operating threshold. Precision (blue) rises monotonically as the threshold increases, progressively eliminating lower scoring candidate pairs. F1 (green) peaks at 1.000 at $\tau = 0.955$ (purple dot) the exact point at which only the three confirmed true clones remain above the threshold and all 28 other candidates have been excluded. The current operating threshold $\tau = 0.85$ (orange dashed line) recovers all three true clones with perfect recall but a precision of 0.097, meaning 28 additional candidate pairs require manual review at this setting.

produce a distinctive near perfect score signature that is easy to identify in practice. Third, the Backstage-integrated pipeline successfully retrieves, parses, and embeds source code from a realistic twelve-service polyglot repository, confirming the end-to-end integration described in Section 5 works correctly on real production code.

4.1.6. Experiment 6 - Controlled Evaluation on the Synthetic Polyglot Ecommerce Repository

4.1.6.1. Experimental Objective

The previous experiments evaluated the pipeline on either a monolingual benchmark (BigCloneBench) or a real world corpus with limited and opportunistically discovered ground truth (Online Boutique). This experiment provides a controlled, fully labelled evaluation on a purpose built synthetic polyglot microservice, designed specifically to test cross language clone detection capability across a range of clone types and language combinations. The objective was to establish a precise baseline measured as F1 per clone group against which the improvement from fine-tuning can be directly quantified in future work.

4.1.6.2. Experimental Setup

The synthetic ecommerce repository comprises five microservices spanning three languages Python (inventory service, order service), JavaScript (notification service, user service), and Java (payment service) and contains six intentionally planted clone groups (A - F). Each group consists of two functions in different services that implement the same algorithm, varying in: the programming language pair (Python - JavaScript, Java - JavaScript, Python - Java, and Java - Java), the degree of structural similarity (nearidentical vs semantically equivalent with different variable names and control flow), and the functional domain (cart calculation, discount application, input validation, pagination, weighted averaging, and retry logic). The full pipeline was run with $\tau = 0.90$, and all 40 cross-service function pairs were evaluated against the labelled ground truth.

4.1.6.3. Results

Figures 4.1.7 presents all three diagnostic panels together. Table 4.1.3 provides the per-clone-group breakdown.

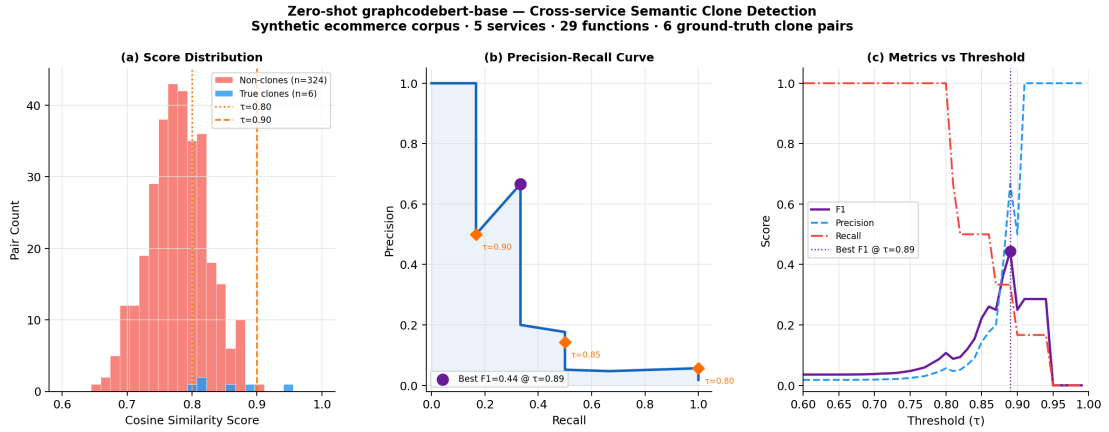


Figure 4.1.7: Zero-shot GraphCodeBERT on the synthetic ecommerce corpus (5 services, 29 functions, 40 cross-service pairs, 6 labelled clone groups A–F). **(a) Score Distribution:** Non clone pairs (red, $n = 324$) form a broad bell centred around 0.78. True clone pairs (blue, $n = 6$) are individually labelled at their score positions and are scattered across the range 0.80 - 0.95. The dotted orange lines mark $\tau = 0.80$ and $\tau = 0.90$; the shaded overlap region between them shows where the two distributions intermingle. Five of the six clone groups fall within this overlap, illustrating the central limitation of zero-shot cross language detection. **(b) Precision-Recall Curve:** The best F1 of 0.44 is achieved at $\tau = 0.89$ (purple dot). At $\tau = 0.90$ (orange diamond), precision = 0.50 and recall = 0.17, reflecting that only the highest-scoring same language clone (Group F, Java–Java) lies clearly above this threshold. Lowering to $\tau = 0.85$ and $\tau = 0.80$ recovers more clones but at rapidly deteriorating precision. **(c) Metrics vs Threshold:** The step like curve shape is expected for a small ground truth of 6 pairs each discrete step represents one additional clone group crossing the threshold. The F1 peak at $\tau = 0.89$ (dotted purple line) sits immediately to the left of the selected operating threshold of 0.90. The abrupt precision spike near $\tau = 0.93$ occurs because above that score only Group F remains, yielding precision = 1.0 but recall = 0.17.

Table 4.1.3: Per-clone-group results on the synthetic ecommerce corpus (zero-shot GraphCodeBERT, $\tau = 0.90$).

Group	Description	Language pair	Score	Detected?
A	Cart total calculation	Python $\leftrightarrow$ JavaScript	≈ 0.85	No
B	Discount application	Java $\leftrightarrow$ JavaScript	≈ 0.85	No
C	Input validation	Python $\leftrightarrow$ JavaScript	≈ 0.85	No
D	Paginated results	Python $\leftrightarrow$ JavaScript	≈ 0.85	No
E	Weighted average	Python $\leftrightarrow$ Java	≈ 0.85	No
F	Retry with backoff	Java $\leftrightarrow$ Java	0.9433	Yes

4.1.6.4. Analysis

The results in Table 4.1.3 and Figure 4.1.7 reveal a consistent and interpretable pattern. Only Group F — a same language Java - Java clone involving retry with exponential backoff logic is detected at the default threshold. All five cross language clone groups (A - E) score in the range 0.83 - 0.87, consistently below $\tau = 0.90$

This result is not a failure of the embedding pipeline but rather a known limitation of the zero-shot mode. Without fine-tuning, the GraphCodeBERT embedding space has not been trained to align semantically equivalent representations across languages. Two functions that implement the same algorithm in Python and JavaScript differ substantially in surface form keyword syntax, identifier conventions, control flow constructs and these surface differences are reflected as a modest angular separation in the embedding space even when the DFG structure is similar. Group F avoids this penalty because both functions are written in Java, and the model has seen sufficient same language Java examples during pre training to produce tightly aligned embeddings for algorithmically similar functions. Panel (a) illustrates this clearly: the five cross language clone groups are indistinguishable from the upper tail of the non clone distribution when viewed by score alone, which is precisely what makes cross language clone detection a hard problem in the zero-shot setting. Panel (b) confirms that even with the optimal threshold of $\tau = 0.89$, the best achievable F1 is only 0.44, driven by the inability to recover Groups A - E without also accepting a large number of non-clone false positives from the overlap region.

4.1.6.5. Conclusion

The synthetic ecommerce experiment serves two purposes. First, it provides a fully labelled, reproducible benchmark for quantifying the improvement attributable to fine-tuning: after fine tuning on cross language pairs from the XLCoST dataset Zhu et al. (2022), the expectation is that Groups A - E will shift from the 0.83 - 0.87 range to scores above 0.90, moving from the overlap region into clear detection

territory. Second, the Group F detection at 0.9433 serves as a sanity check: the pipeline correctly identifies a same language semantic clone that is structurally close but not textually identical, confirming that the DFG guided embedding is working as intended and that the zero-shot failure on cross-language pairs is a threshold and alignment problem rather than a fundamental embedding failure. The post fine tuning evaluation against this ground truth is described in Section 6.3.

4.1.7. Experiment 7 - Fine-Tuned GraphCodeBERT on Unsliced BigCloneBench Subset

4.1.7.1. Experimental Objective

This experiment evaluates the effect of supervised fine-tuning on a 100,000-pair subset of BigCloneBench. The primary aim is to determine how much task-specific adaptation improves discriminative performance when the model is provided with full, unmodified function bodies alongside their extracted DFG representations. This experiment also serves as the reference baseline for Experiment 8, which investigates whether DFG-guided slicing of long functions yields further improvement.

4.1.7.2. Experimental Setup

Fine-tuning was performed on `microsoft/graphcodebert-base` using a 100,000-pair sample drawn from the BigCloneBench dataset via the CodeXGLUE benchmark Lu et al. (2021). Functions were used in their original, unmodified form with no preprocessing beyond tokenization; no slicing was applied. Data Flow Graphs were extracted from each function using tree-sitter. The input configuration followed the settings reported in Guo et al. (2021): a maximum code token length of 256 and a maximum DFG node count of 64. Functions whose BPE-projected token count exceeded the 256-token budget were silently truncated by the tokeniser rather than compressed prior to encoding.

The model was trained using a dual-encoder architecture with a classification head. Training used the AdamW optimiser with a linear learning rate schedule and standard cross-entropy loss over binary clone labels. The similarity threshold for binary prediction was selected by searching over candidate values on the *validation* set only, to avoid data leakage into the test evaluation. The threshold of 0.65 was identified as optimal on this basis and applied unchanged to the held-out test set.

4.1.7.3. Results

Evaluation was conducted on the standard 20,000 split of BigCloneBench. Results are summarised in Table 4.1.4.

Table 4.1.4: Fine-tuned GraphCodeBERT (unsliced, 100k subset) on BigCloneBench test set of 20,000.

Metric	Score
Accuracy	0.8864
Precision	0.8736
Recall	0.9036
F1 Score	0.8884

4.1.7.4. Analysis and Conclusion

Fine-tuning yields a dramatic and consistent improvement over the zero-shot baseline across all metrics.

These gains confirm that the pre-trained GraphCodeBERT representations contain substantial latent semantic signal that is not accessible without task-specific adaptation. Fine-tuning with DFG-augmented inputs teaches the model to align its embedding space with the clone detection objective, enabling meaningful cosine similarity comparisons between function pairs.

Nevertheless, the tokenizer discards tail tokens regardless of their semantic importance, potentially causing the model to lose critical data-flow information that would otherwise distinguish clone pairs. This motivates the DFG-guided slicing strategy evaluated in Experiment 8, which replaces truncation with a semantics-preserving compression step designed to retain the most data-flow-relevant lines

within the token budget.

4.1.8. Experiment 8 - Fine-Tuned GraphCodeBERT on DFG-Sliced BigCloneBench Subset

4.1.8.1. Experimental Objective

Experiment 7 demonstrated that truncation of long functions is a likely source of performance loss, as the tokenizer discards tail tokens without regard for their semantic importance. This experiment investigates whether replacing truncation with the DFG-guided line-based slicing procedure described in the Methodology Section yields measurable improvement on the same 100,000-pair BigCloneBench subset. The slicing procedure is designed to preserve the data-flow-relevant core of each function within the token budget, rather than blindly cutting at a fixed token boundary.

4.1.8.2. Experimental Setup

The experimental configuration is identical to Experiment 7 in all respects, with one change to the preprocessing pipeline. Functions whose BPE-projected token count exceeded the 256-token budget were compressed prior to encoding using the DFG-guided slicing procedure. Specifically, each DFG token index was mapped to its corresponding source line; the union of all lines referenced by DFG definition was formed; this set was expanded by one context line above and below each referenced line to preserve local syntactic coherence and a reduced source text was reconstructed from only the retained lines. The resulting slice was reparsed and its DFG reextracted, yielding a compact representation that retains the function’s core data-flow logic while discarding logging statements, comments, and structural boilerplate Alomari & Stephan (2022), Lekssays et al. (2025), Weiser (1984). Functions already within the token budget were passed through unchanged.

As in Experiment 7, the similarity threshold was selected by searching over candidate values on the validation set only. A threshold of 0.70 was identified as

optimal and applied unchanged to the held-out test set.

4.1.8.3. Results

Results on the standard 20,000 split are summarized in Table 4.1.5.

Table 4.1.5: Fine-tuned GraphCodeBERT (DFG-sliced, 100k subset) on Big-CloneBench test set of 20,000.

Metric	Score
Accuracy	0.9223
Precision	0.9089
Recall	0.9387
F1 Score	0.9236

4.1.8.4. Analysis and Conclusion

The DFG-guided slicing procedure produces consistent improvements across all four metrics relative to the unsliced baseline of Experiment 7.

The improvement is consistent with the theoretical motivation of the slicing procedure. Truncation at a fixed token boundary may discard variable assignments, return expressions, or conditional branches that are essential to distinguishing a semantic clone from a non-clone pair. The slicing procedure, by contrast, retains lines that participate in data-flow definitions or uses, which are precisely the structural features that GraphCodeBERT’s DFG-augmented pretraining is designed to exploit Guo et al. (2021). By ensuring that the model receives a semantically coherent representation of each function rather than an arbitrarily truncated one, the slice allows the DFG encoder to construct more faithful function embeddings.

The results demonstrate that the slicing strategy is effective and additive over fine-tuning alone, motivating its retention in the final pipeline.

4.2. Function Relocation Analyzer

4.2.1. Experiment 1 - Evaluation of Distributed Tracing Integration Strategies

4.2.1.1. Experimental Objective

The objective of this experiment was to find a tracing configuration that was effective, stable, and maintainable for the Function Relocation Analyzer (FRA) plugin within the Backstage ecosystem. Since there are multiple tracing tools such as OpenTelemetry, Jaeger, Zipkin, and Prometheus that provide similar yet differentiated functionality, a comparative analysis was necessary to find a tracing configuration that was "most compatible" with collecting distributed trace data within the Node.js plugin-based backend of Backstage. This experiment was a foundational element for the FRA pipeline. The accuracy of trace data was a key determinant for the quality of trace preprocessing, function call analysis, and relocation decision logic. A poorly configured tracing layer would taint all downstream analysis.

4.2.1.2. Experimental Setup

The experiments were carried out iteratively, starting with the uninstrumented baseline and then gradually adding tracing layers. The experiments were assessed based on instrumentation coverage, metadata quality, integration complexity, and compatibility with the internal architecture of Backstage. The important parameters of the experimental setup are listed below.

With a vendor-neutral observability standard (OpenTelemetry) used as a data collection and export layer, and a dedicated tracing backend for visualisation (Jaeger), a better solution will be achieved in terms of instrumentation, metadata, and integration maintainability compared to native client integrations of each tool.

Parameter	Details
Platform	Backstage (Spotify open-source developer portal, CNCF)
Runtime environment	Node.js backend with plugin-based architecture
Tracing tools evaluated	OpenTelemetry (OTel), Jaeger, Zipkin, Prometheus
Evaluation criteria	Instrumentation coverage, metadata richness, integration effort, maintainability, compatibility with existing Backstage observability
Goal	Achieve automated service and function tracing visible through the Backstage interface, with minimal code modification and reliable data export

Figure 4.2.1: Experimental environment parameters for the tracing integration study.

4.2.1.3. Experimental Procedure

The six variants were assessed sequentially. Each was a refinement on what had been found previously. The variants are presented in the following sections in the order that they were conducted.

Variant 1 - Baseline (No Instrumentation)

The first step was to create a control baseline, which did not have any tracing configuration. This ensured that the services provided by Backstage were working normally before introducing any additional dependencies. It provided a measure of the overhead introduced by the different variants.

- **Observation:** No runtime overhead or performance degradation was observed.
- **Limitation:** No information about relationships between services or function calls was available.

Conclusion: A stable control environment was verified; however, no tracing data was available for use by the Function Relocation Analyzer (FRA).

Variant 2 – Direct Jaeger Client Integration

Jaeger client instrumentation was directly incorporated into the Backstage backend by adding the Jaeger library as a package dependency and exporting traces to `http://localhost:16686`.

- **Outcome:** Jaeger successfully received trace spans from the backend. There was a lack of instrumentation of internal function calls, which needed to be individually wrapped.
- **Challenge:** Each function needed to be individually and manually wrapped. This led to an increase in code complexity.

Conclusion: Although it was a working solution, it was not a scalable solution. The direct Jaeger client integration would not be suitable in a plugin-based architecture.

Variant 3 – Direct Zipkin Client Integration

After the trial with Jaeger, Zipkin was used as a different direct exporter. A Zipkin dependency was added, and the backend was set up to forward the traces to a local Zipkin server.

- **Result:** Exporting traces was successful, though the metadata was limited compared to Jaeger. The Zipkin client was not naturally compatible with the automatic instrumentation of the Node.js functions.
- **Challenge:** Integration with the internal Backstage observability pipeline or existing Prometheus metrics was difficult. Correlation between cross-service calls was not as strong as Jaeger's implementation using the span approach.

Conclusion: Zipkin's native client was not compatible with the FRA objective of effortless automatic instrumentation and cross-service call correlation. Not a viable option.

Variant 4 - OpenTelemetry Standalone Instrumentation

OpenTelemetry (OTel) was added to Backstage as a standalone SDK, which did not connect to any external visualisation backend. Three packages were added as dependencies: @opentelemetry/sdk-node, @opentelemetry/auto-instrumentations-node, and @opentelemetry/exporter-prometheus. A specific script, instrumenta-

tion.js, was created to initialise the OpenTelemetry SDK during backend startup using a call to `start()`.

- **Outcome:** The automatic instrumentation process was successful across all components in the backend, with no need for manual wrapping of functions. The Prometheus exporter provided real-time metrics, and trace spans were successfully created and validated using the console exporter provided by OpenTelemetry.
- **Limitation:** At this point in time, there was no external trace visualisation available, only trace data in console output.

Conclusion: OpenTelemetry was found to be the most flexible and modern solution for automatic trace integration into Backstage. OpenTelemetry was identified as the core instrumentation layer to build upon.

Variant 5 — OpenTelemetry with Jaeger Collector (Adopted Configuration)

The OTel SDK from Variant 4 was connected to a Jaeger backend through an OpenTelemetry Collector. Crucially, no Jaeger SDK dependency was included in the application code. Instead, the Collector was used as a middleman, with OTel exporting spans to the Collector, which then routed them to Jaeger at `http://localhost:16686`. As such, this configuration was achieved solely through code, with no changes to the application code.

The changes were as follows:

- The `app-config.yaml` file was updated to include a `jaegerQueryUrl`, as well as a proxy endpoint for plugin-level communication at `/jaeger`.
- No Jaeger SDK dependency was included in `package.json`, with this responsibility instead falling to the OTel Collector.
- The OTel part of the configuration within `app-config.yaml` was updated for plugin-level communication.

- **Outcome:** Stable trace visualisation within Jaeger. Minimum changes required at a code level. Fully compatible with Prometheus metrics exporter from Variant 4.

Conclusion: This variant provided the best balance of all four criteria for evaluation.

Variant 6 - Using OpenTelemetry with Zipkin Backend

As a sixth comparative validation, the OTel Collector was reconfigured to send the traces to a Zipkin backend instead of Jaeger, using the same OTel instrumentation layer as in Variant 5. This was done to evaluate if the advantage enjoyed by Variant 5 was a result of the OTel instrumentation layer itself, rather than the Jaeger backend.

- **Outcome:** Technically functional. The traces are correctly exported and received. However, the Zipkin interface does not have the same level of query ability for span relationships or the depth of the service dependency graph as Jaeger.

Conclusion: OTel with Zipkin is technically valid but substandard for the required diagnostic depth for FRA. Not adopted.

4.2.1.4. Results

Figure 4.2.2 summarizes the findings across all six variants against the four evaluation criteria.

4.2.1.5. Adopted Configuration

After all six versions were finished, the FRA tracing infrastructure chose the OpenTelemetry with Jaeger Collector configuration (Variant 5). Below is a list of all the adopted stacks.

Variant	Auto coverage	Metadata richness	Integration effort	Cross-service correlation	Adopted?
Baseline	None	None	None	None	No (control)
Direct Jaeger	Partial (manual)	Moderate	High	Moderate	No
Direct Zipkin	Low	Low	High	Low	No
OTel standalone	Full (automatic)	High	Low	N/A (no backend)	No (foundation)
OTel + Jaeger ✓	Full (automatic)	High	Low	High	YES
OTel + Zipkin	Full	Moderate	Low	Moderate	No

Figure 4.2.2: Comparative evaluation of tracing configurations across the six experimental variants. The adopted configuration (OTel + Jaeger) is highlighted.

Component	Configuration	Role in FRA
OpenTelemetry	@opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node @opentelemetry/exporter-prometheus instrumentation.js initialises SDK at startup	Automatic instrumentation of all backend components; Prometheus metrics export; trace span generation
Jaeger (via OTel Collector)	jaegerQueryUrl: http://localhost:16686 Proxy endpoint: /jaeger in app-config.yaml	Trace visualisation, span relationship querying, and service dependency graph via Jaeger UI
Zipkin	Not configured. No exporter present. Mentioned in app-config for potential future extension.	Not used in current implementation

Figure 4.2.3: Adopted tracing stack for the Function Relocation Analyzer.

4.2.1.6. Discussion and Conclusion

The hypothesis was verified: the vendor-neutral OTel layer with a Jaeger visualisation backend was better than all direct native client integrations for each measure. The key architectural takeaway from the above experiment is the separation of concern between instrumentation and the backend visualisation layer. The FRA gains two important advantages from using the OTel Collector as an intermediary: instrumentation code in the Backstage backend is backend agnostic – changing from Jaeger to any other backend that is compatible with the vendor-neutral OTel

layer (Tempo or Zipkin) only requires a change to the configuration files; automatic instrumentation with @opentelemetry/auto-instrumentations-node provides comprehensive instrumentation of all backend components without the need for manual wrapping at the function level, which is a prerequisite for the trace pre-processing phase of the FRA to capture all function calls. Variants 2 and 3, which used a direct native client approach, did not succeed in meeting the coverage criterion because manual instrumentation would not be able to ensure all function calls are properly detected, which would mean systematic blind spots in the analysis performed by the FRA. OpenTelemetry standalone succeeded in meeting the coverage criterion but failed to meet the persistent query store criterion, which was necessary for meeting the batch analysis criterion. Only Variant 5 succeeded in meeting all four criteria. The adoption of this particular configuration is referenced in the implementation chapter of the paper (Section 5.X), which discusses how the adopted tracing solution was incorporated in the deployed plugin of the FRA.

Chapter 5

Implementation

Chapter 6

Evaluation

Bibliography

- Abdelfattah, A. S., Rodriguez, A., Walker, A. & Cerny, T. (2023), ‘Detecting semantic clones in microservices using components’, *SN Computer Science* **4**(5), 470.
- Akram, J., Shi, Z., Mumtaz, M. & Luo, P. (2018), Dccd: An efficient and scalable distributed code clone detection technique for big code, *in* ‘Proceedings of the 30th International Conference on Software Engineering and Knowledge Engineering (SEKE 2018)’.
- Alomari, H. W. & Stephan, M. (2022), ‘srcClone: Clone detection based on program slicing’, *Journal of Systems and Software* .
- Barua, B. & Kaiser, M. S. (2024), ‘Ai-driven resource allocation framework for microservices in hybrid cloud platforms’, *arXiv preprint arXiv:2412.02610* .
- Bhatia, R. (2024), ‘Optimizing microservice communication and exploring hybrid architectures: Leveraging service mesh solutions and combining microservices with soa’, *International Research Journal of Engineering and Technology (IRJET)* **11**(7).
- Brunsfeld, M. et al. (n.d.), ‘Tree-sitter: An incremental parsing system for programming tools’.
URL: <https://tree-sitter.github.io>
- Cerny, T., Abdelfattah, A. S., Al Maruf, A., Janes, A. & Taibi, D. (2023), ‘Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study’, *Journal of Systems and Software* **206**, 111829.
- Cerny, T., Svacina, J., Das, D., Bushong, V., Bures, M., Tisnovsky, P., Frajtek, K., Shin, D. & Huang, J. (2020), ‘On code analysis opportunities and challenges for enterprise systems and microservices’, *IEEE access* **8**, 159449–159470.

- Devlin, J. et al. (2019), BERT: Pre-training of deep bidirectional transformers for language understanding, *in* 'NAACL'.
- Faustino, D., Gonçalves, N., Portela, M. & Silva, A. R. (2024), 'Stepwise migration of a monolith to a microservice architecture: Performance and migration effort evaluation', *Performance Evaluation* **164**, 102411.
- Feng, Z. et al. (2020), 'CodeBERT: A pre-trained model for programming and natural languages'.
- Google (n.d.), 'Online boutique: Cloud-native microservices demo application'.
URL: <https://github.com/GoogleCloudPlatform/microservices-demo>
- Guo, D. et al. (2021), GraphCodeBERT: Pre-training code representations with data flow, *in* 'ICLR'.
- Kang, J., Wu, X., Wu, H., Yang, D. & Dai, W. (2023), Automatic microservice orchestration and deployment method based on the modular type package for industrial edge applications, *in* '2023 IEEE 32nd International Symposium on Industrial Electronics (ISIE)', IEEE, pp. 1–6.
- Lekssays, A. et al. (2025), 'LLMxCPG: Leveraging large language models with code property graphs'.
- Lewis, J. & Fowler, M. (2014), 'Microservices'.
- Liu, Y. et al. (2019), 'RoBERTa: A robustly optimized BERT pretraining approach'.
- Lu, S. et al. (2021), 'CodeXGLUE: A machine learning benchmark dataset for code understanding and generation'.
- Mo, R., Zhao, Y., Feng, Q. & Li, Z. (2021), The existence and co-modifications of code clones within or across microservices, *in* 'Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)'.

- Newman, S. (2019), *Monolith to microservices: evolutionary patterns to transform your monolith*, O'Reilly Media.
- Nobre, J., Pires, E. S. & Reis, A. (2023), 'Anomaly detection in microservice-based systems', *Applied Sciences* **13**(13), 7891.
- Nogueira, V. L., Felizardo, F. S., Amaral, A. M., Assunção, W. K. & Colanzi, T. E. (2024), Insights on microservice architecture through the eyes of industry practitioners, in '2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)', IEEE, pp. 765–777.
- Noor, M., Ibrahim, R., Sahid, N. M. Z. & Huang, S. (2024), 'Comparison of cloud computing, saas, soa and microservices', *Journal of Advanced Research in Applied Sciences and Engineering Technology* .
- Panichella, S., Rahman, M. I. & Taibi, D. (2021), Structural coupling for microservices, in 'Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)'.
- Ramu, V. (2023), 'Performance impact of microservices architecture', *Rev. Contemp. Sci. Acad. Stud* **3**(6).
- Somashekar, G. & Gandhi, A. (2021), Towards optimal configuration of microservices, in 'Proceedings of the 1st Workshop on Machine Learning and Systems', pp. 7–14.
- Svajlenko, J. & Roy, C. K. (2015), Evaluating clone detection tools with big-clonebench, in '2015 IEEE international conference on software maintenance and evolution (ICSME)', IEEE, pp. 131–140.
- Thönes, J. (2015), 'Microservices', *IEEE software* **32**(1), 116–116.
- Thongtan, T. & Phienthrakul, T. (2019), Sentiment classification using document embeddings trained with cosine similarity, in 'ACL Student Research Workshop'.

- Tuusjärvi, K., Kasurinen, J. & Hyrynsalmi, S. (2024), How does migration to microservices happen? a survey of the finnish software industry., *in* ‘TKTP’, pp. 22–32.
- Walker, A., Das, D. & Cerny, T. (2020), ‘Automated code-smell detection in microservices through static analysis: A case study’, *Applied Sciences* **10**(21), 7800.
- Wang, W., Li, G., Ma, B., Xia, X. & Jin, Z. (2020), Detecting code clones with graph neural network and flow-augmented abstract syntax tree, *in* ‘2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)’, IEEE, pp. 261–271.
- Weiser, M. (1984), ‘Program slicing’, *IEEE Transactions on Software Engineering* **10**(4).
- Yamaguchi, F. et al. (2014), Modeling and discovering vulnerabilities with code property graphs, *in* ‘IEEE S&P’.
- Zhu, M. et al. (2022), XLCOST: A benchmark dataset for cross-lingual code intelligence.